Lecture 6 · Wed 10/14/2026

CS160 Compilers — Fall 2026

Parsing I.

Yu Feng

Paper and pencil out — two of today's exercises are Midterm 1 questions.

Where we are.



Monday: characters → tokens, with a DFA.

Today: tokens → tree. Every phase after today reads the tree and never looks at your text again.

The same tokens can mean two different trees.

```
tokens:   INT(1)  OP(-)  INT(2)  OP(-)  INT(3)

tree A:   (1 - 2) - 3   =  -4
tree B:   1 - (2 - 3)   =   2
```

Nothing in the token list says which tree is meant. Python picks A — but that is a *choice*, not something written in the string.

A DFA cannot even check that parentheses balance: it has a fixed number of states, so it cannot count. Tokens → tree needs a different tool.

That tool is a grammar: a written-down decision about which trees exist. Today is about reading one, fixing one, and turning one into a program that finds the tree.

Which tree does this string mean — and who decides?

By the end you can take a grammar and a string and produce the derivation and the parse tree, by hand.

You can say whether a grammar is *ambiguous* and rewrite it so that precedence and associativity are forced by its shape.

You can turn a grammar into a top-down parser, rule by rule, and trace it by hand.

All three are on Midterm 1, Monday. The third is also what you are building this week.

A grammar is a set of rewrite rules.

```
E -> E '-' T  |  T
T -> T '*' F  |  F
F -> INT  |  '(' E ')'
```

`E, T, F` — **nonterminals**: things that get rewritten.

`'-' '*' '(' ')' INT` — **terminals**: tokens; nothing rewrites them.

`E` — the **start symbol**.

production

one rule: one nonterminal on the left, one right-hand side. `E -> E '-' T | T` is two productions sharing a left side, written with `|` for “or”.

terminal

a token *kind*. `INT` is any integer token; `'-'` is the operator token whose text is `-`. The parser asks `tok.kind == "INT"` and never looks at the characters — the lexer already did.

Read `E -> E '-' T` as: an E can be an E, a minus, a T. The language is every token string you can reach from E.

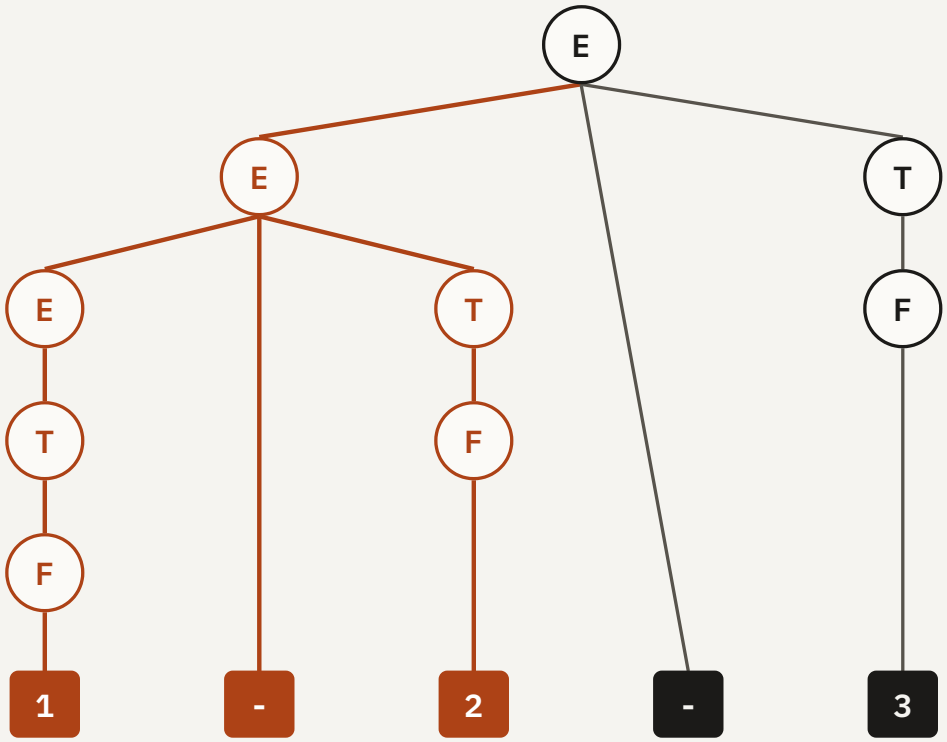
Rewrite the leftmost nonterminal until only tokens remain.

```
E  =>  E - T  =>  E - T - T  =>  T - T - T  =>  F - T - T  =>  1 - T - T
   =>  1 - F - T  =>  1 - 2 - T  =>  1 - 2 - F  =>  1 - 2 - 3
```

derivation a sequence of rewrites; read => as “rewrites to” — one rule applied. Not the same arrow as the `->` inside a rule.

Leftmost: each step rewrites the leftmost nonterminal, so the only thing left to choose is *which* rule. Here the first choice was `E -> E '-' T`, applied twice.

A parse tree is that derivation, drawn.



parse tree

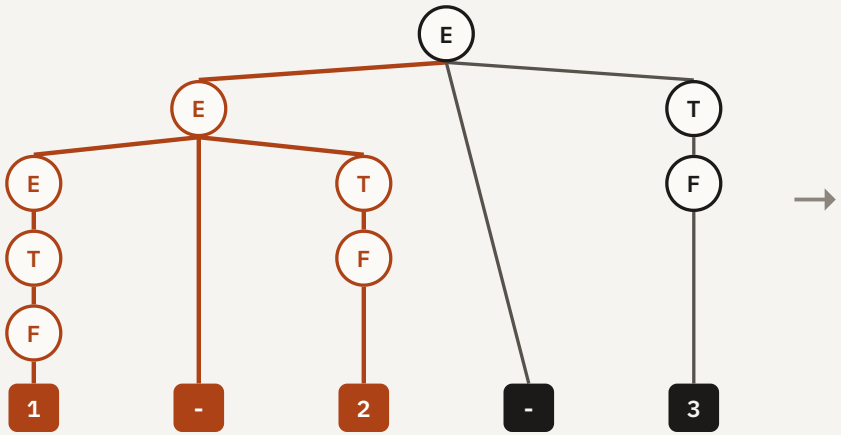
one node per rule application: the nonterminal, with its right-hand side as its children. Read the leaves left to right and you get the input back.

Look at the shape: the left `-` sits *under* the right `-`. This tree is `(1 - 2) - 3`. The grammar chose tree A.

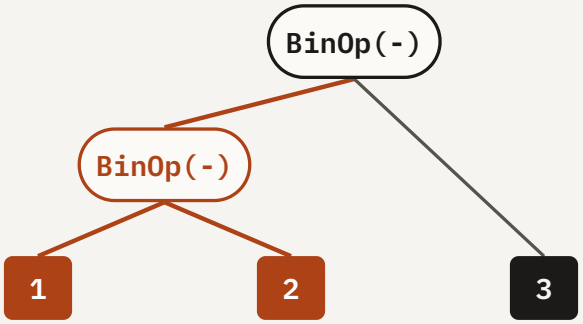
nonterminal token

The AST is the parse tree with the grammar's bookkeeping thrown away.

parse tree · 14 nodes



AST · 5 nodes



The parse tree records *how* the grammar matched. The **AST** — *abstract syntax tree* — records *what was said*.

Chains like E → T → F → 1 collapse to the leaf; parentheses vanish, because the shape already carries them.

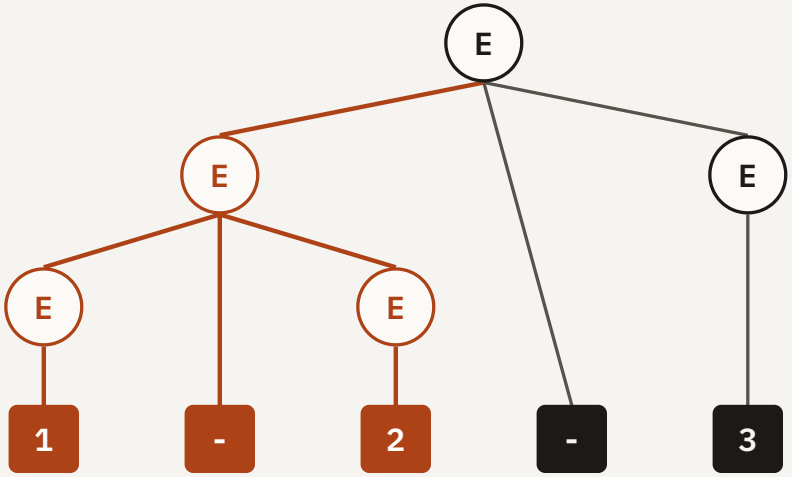
Lecture 3's compiler consumes the AST and never sees a parse tree.

```
$ python3 -c "import ast; print(ast.dump(ast.parse('1 - 2 - 3', mode='eval').body))"
BinOp(left=BinOp(left=Constant(value=1), op=Sub(), right=Constant(value=2)), op=Sub(), right=Constant(value=3))
```

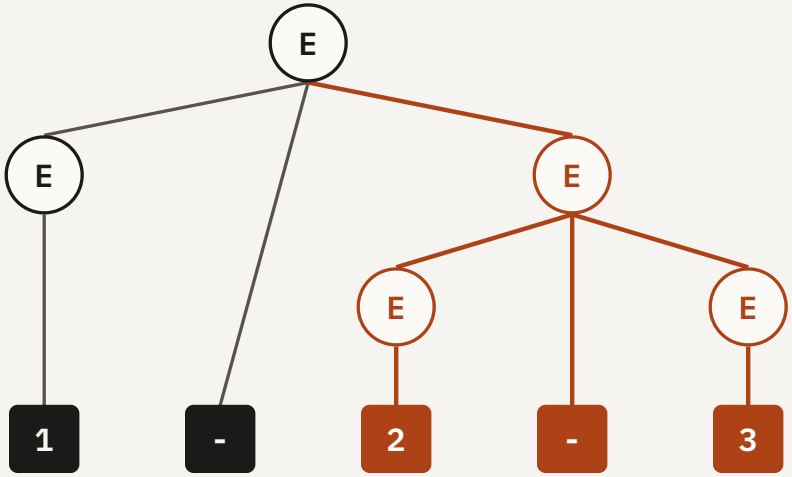
That is the right-hand tree, printed by Python. **The parser we build today has to produce exactly this.**

Two trees for one string.

```
E -> E '-' E  |  INT
```



(1 - 2) - 3 = −4



1 - (2 - 3) = 2

This one-line grammar also generates 1 - 2 - 3 — in two ways. A grammar with two parse trees for one string is **ambiguous**: it has not decided, so the parser will decide for you, and you will not know how.

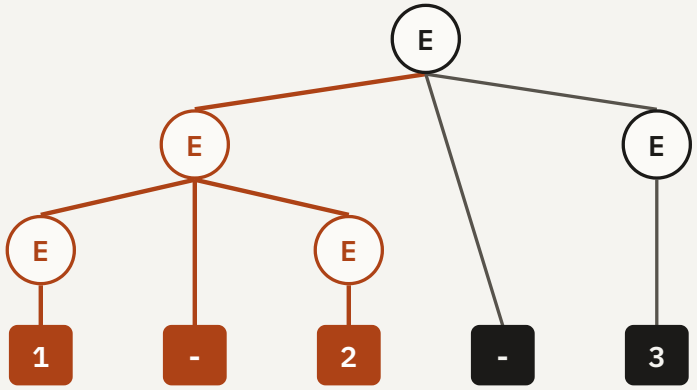
Slide 5's grammar gives exactly one tree. The next two slides say what it did differently.

Associativity: which side recurses.

associativity when the *same* operator repeats, which way it groups. `(1 - 2) - 3` is left-associative; `1 - (2 - 3)` is right-associative.

```
E -> E '-' T  |  T
```

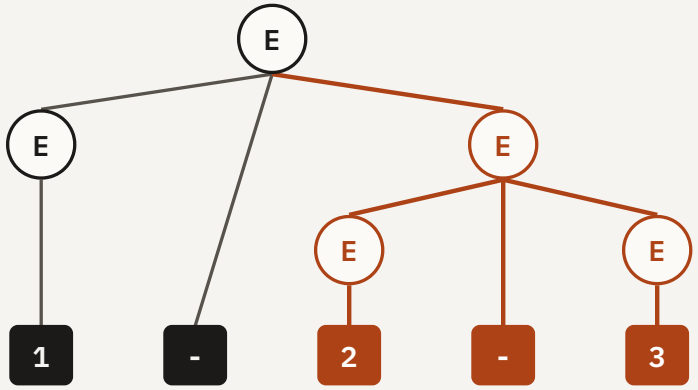
recurses on the **left** · what Python does



`(1 - 2) - 3`

```
E -> T '-' E  |  T
```

recurses on the **right**



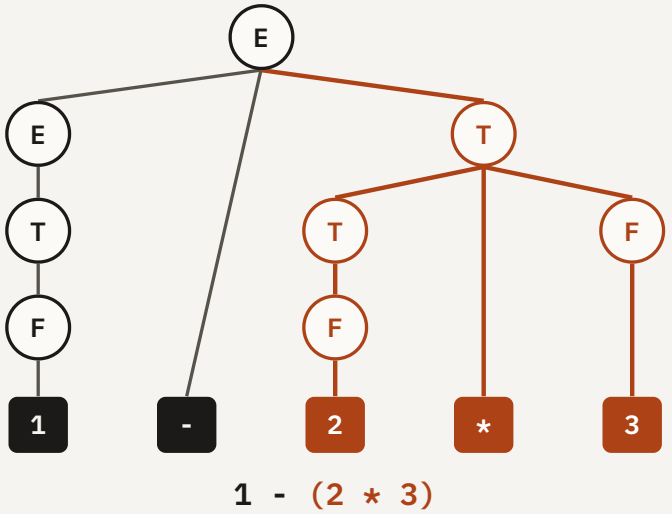
`1 - (2 - 3)`

The side that recurses is the side that keeps growing. **Left-recursive rule, left-associative operator.** One rule, one tree: the ambiguity is gone.

Precedence: one nonterminal per level.

precedence when two *different* operators compete for an operand, which one gets it. `*` binds tighter than `-`, so `1 - 2 * 3` is `1 - (2 * 3)`.

```
E -> E '-' T  |  T       level 1: '-'   (loosest)
T -> T '*' F  |  F       level 2: '*'
F -> INT  |  '(' E ')'   level 3: atoms (tightest)
```



`-` lives only in E, `*` only in T. To get a `-` underneath a `*` you would need T to reach E — and it can't, except through parentheses.

Precedence, enforced by shape.

So slide 5's grammar was built this way on purpose: one nonterminal per precedence level, each recursing on the left. That is the whole recipe.

The same language, written so that each string has one tree.

```
ambiguous                unambiguous

E -> E '-' E             E -> E '-' T  |  T          '-' lives here   (binds loosest)
   | E '*' E             T -> T '*' F  |  F          '*' lives here   (binds tighter)
   | INT                 F -> INT  |  '(' E ')'     the operands
   | '(' E ')'
```

Precedence. Give each operator its own nonterminal, the loosest operator at the top. An operand of `-` is a T, and a T contains no `-` — so `*` is always grouped first.

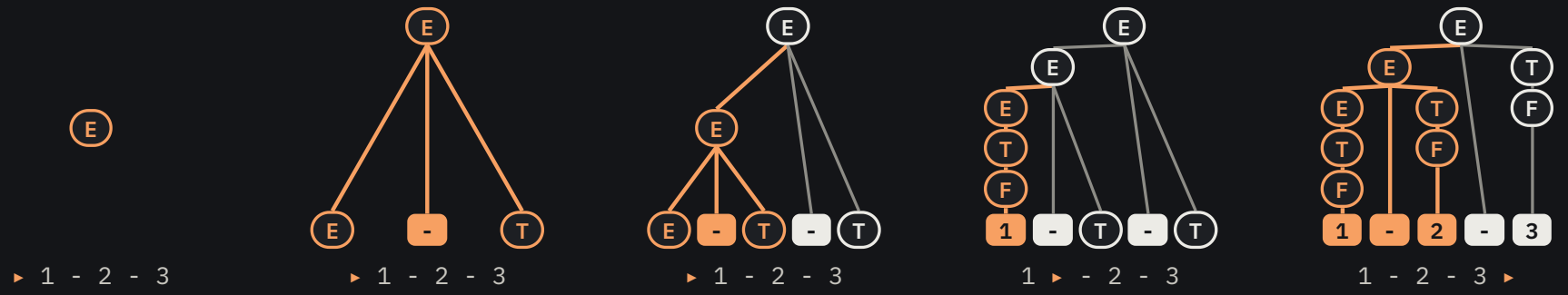
Associativity. In each operator's rule, recurse on the side it groups toward: `E -> E '-' T`, not `E -> T '-' E`, so `1 - 2 - 3` is `(1 - 2) - 3`.

Operands. Numbers and parenthesized expressions go at the bottom. `'(' E ')'` leads back to E, so inside parentheses anything is allowed again.

That settles *which* tree. Now the other half of today: given the tokens, how does a program *find* it?

Start at the root. Grow the tree toward the leaves. Match the leaves against the input.

Top-down parsing: begin with the start symbol as the root. Repeatedly pick the leftmost nonterminal at the bottom edge of the tree, choose one of its productions, and hang its right-hand side underneath. Whenever a terminal appears at the bottom edge, it must equal the next input token — consume it and move on.



tree so far	input	action
E	▸ 1 - 2 - 3	expand E with E -> E '-' T
E -> E - T	▸ 1 - 2 - 3	expand left E with E -> E '-' T
... E -> T	▸ 1 - 2 - 3	expand T with T -> F, then F -> INT
... F -> 1	1 ▸ - 2 - 3	leaf 1 matches input: consume
... '-'	1 - ▸ 2 - 3	leaf '-' matches input: consume

This is slide 6's leftmost derivation, performed by a program that reads the input left to right. The order of the steps is fixed. The only freedom is the same one as on slide 6: *which production to choose.*

One token of lookahead decides — if the grammar lets it.

The general answer is trial and error: choose a production, and if the input later refuses it, back up and try the next one. Correct, slow, and nobody does it.

The practical answer: look at the **next token** before choosing — one token of **lookahead**. If each production of a nonterminal starts with a different token, one look decides. A parser that chooses this way is a **predictive parser**; next Wednesday we will call the grammars it works on LL(1).

`F -> INT | '(' E ')'`	next token `INT` → first. Next token `'('` → second.	**one look decides ✓**
`E -> E '-' T | T`	next token `INT(1)` under **both**. One look does not decide.	✗

E has a problem that no amount of lookahead fixes, and it has a name. Next slide.

A rule that starts with itself.

A nonterminal A is **left-recursive** if it can derive a string that begins with A itself. The common case — **direct left recursion** — is a production whose right-hand side begins with its own left-hand side:

`A -> A α | β`	α, β : any sequence of grammar symbols; β does not begin with A
`E -> E '-' T | T`	our E: α = `'-' T`, β = `T` — **direct left recursion**

Why it kills top-down parsing: at E, choose `E -> E '-' T`. To parse it, first parse an E. Same nonterminal, same input position, same choice — the parser expands E forever and never consumes a token.

```
E  =>  E '-' T  =>  E '-' T '-' T  =>  E '-' T '-' T '-' T  =>  ...
▸ 1 - 2 - 3    cursor never moves
```

This is not a defect in the grammar: slide 10 chose left recursion *on purpose*, for left associativity. It is a mismatch between this grammar and top-down parsing. So we rewrite the grammar into one that generates the same strings without starting with itself.

Move the recursion to the right, and give the rest a name.

the general rule · EaC 3.3 / Dragon 4.3.3

```
A  ->  A α1 | ... | A αm | β1 | ... | βn

becomes

A  ->  β1 A' | ... | βn A'
A' ->  α1 A' | ... | αm A' | ε
```

A now starts with a β — never with A. A' appends one α at a time, as many as wanted.

ε (epsilon) : the empty right-hand side — A' may also produce nothing.

our E · α = '-' T, β = T

```
E -> E '-' T | T

becomes

E  ->  T E'
E' ->  '-' T E'  |  ε
```

T gets the same treatment with '*' and F.

the same string, both ways

```
old:   E  =>  E - T  =>  E - T - T      =>  T - T - T
new:   E  =>  T E'   =>  T - T E'  =>  T - T - T E'  =>  T - T - T
```

Same strings — T, T - T, T - T - T, ...: a T, then any number of '-' T. Only the shape of the derivation changed: the old one grows on the left, the new one on the right.

Hand-written parsers write E' as a repetition instead of a helper nonterminal: E -> T ('-' T)*, where **(...)*** means *the group, zero or more times*. Same rule; from here on we use this form.

Now E begins with T, and E''s only decision is "is the next token '-'?" — one look decides.

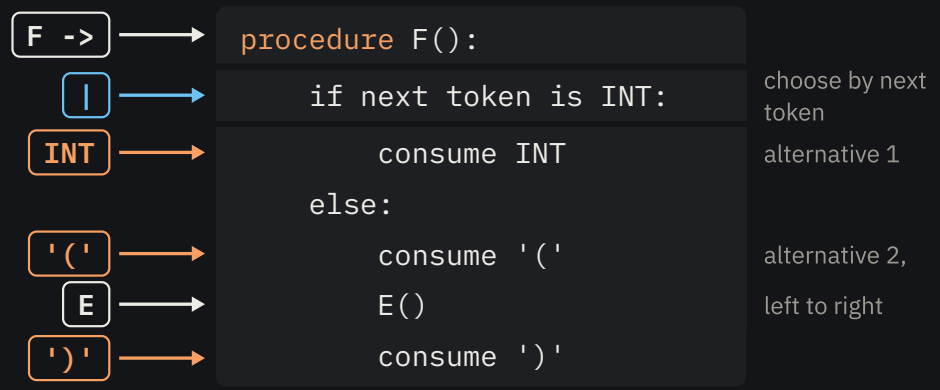
Each nonterminal becomes one procedure. Calling it means: “read one of these, starting at the cursor.”

Write a procedure for every nonterminal, named after it: `E()`, `T()`, `F()`. The contract of `F()`: when called, the cursor is at the start of some F; it reads exactly that F, and returns with the cursor just past it. `E()` and `T()` promise the same. **Consume** means: check that the next token is the one expected, then move the cursor past it.

in the grammar	in the procedure
`X` a nonterminal	`call X()` reads one X and moves the cursor past it
`'x'` a terminal	`consume 'x'` next token must be 'x'; move past it; else error
`a | b` a choice	`look at the next token` take the alternative it starts
`( … )*` a repeat	`while next starts the group:` do the group, until the next token doesn't start it

F, symbol by symbol · every symbol of the rule becomes one line

```
F -> INT | '(' E ')'
```

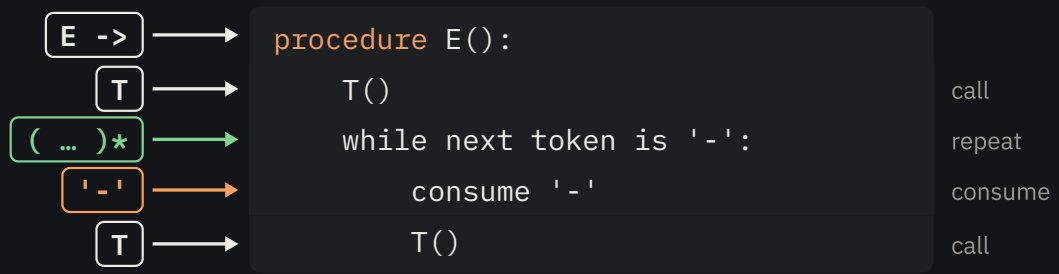


Read alternative 2 left to right: `'('` is a terminal → consume it. E is a nonterminal → call `E()`, which reads the whole expression inside the parentheses, however long, and comes back with the cursor on the `')'`. `')'` → consume it. `F()` has now read exactly one F.

E has a loop in its rule, so E() has a loop in its body. Three procedures are the whole parser.

E, symbol by symbol · colours are slide 17's table rows

```
E -> T ( '-' T )*
```



To parse an input: call `E()`. If it returns with the cursor at the end of the input, the input is an E; if any consume fails, it isn't.

One procedure per nonterminal, calling each other, is **recursive descent**. *Descent*: `E()` calls `T()` calls `F()`, down the grammar. *Recursive*: `F()` calls `E()` for parentheses, back up to the top.

The calls nest like the chains in slide 7's tree: the leaf `1` sits under `E → T → F`, and it is consumed by `F()` running inside `T()` running inside `E()`.

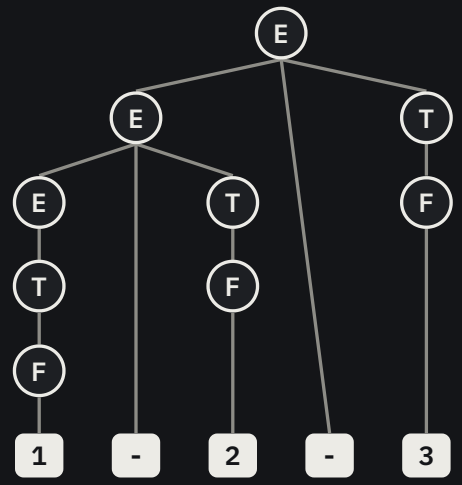
`T -> F ( '*' F )*`	→	`procedure T(): F(); while next token is '*': consume '*'; F()`	
`F -> INT | '(' E ')'`	→	`procedure F():`	slide 17

Trace: E() on 1 - 2 - 3

```
E -> T ( '-' T )*                T -> F ( '*' F )*                F -> INT | '(' E ')'
E(): T(); while '-': consume; T()   T(): F(); while '*': consume; F()   F(): INT? consume : ( E() )
```

indentation = call depth · right column = input, cursor at ▸ ◀ step 9 / 9 ▶

E() starts. rule T ('-' T)*. first: call T()	▸ 1 - 2 - 3
T() starts. rule F ('*' F)*. first: call F()	▸ 1 - 2 - 3
F() starts. next is INT → consume 1. F() returns.	1 ▸ - 2 - 3
T() continues. next is '-', not '*' → loop skipped. T() returns.	1 ▸ - 2 - 3
E() continues. next is '-' → consume '-'; call T()	1 - ▸ 2 - 3
T() → F() consumes 2; T sees '-', not '*'; T() returns	1 - 2 ▸ - 3
E() continues. next is '-' → consume '-'; call T()	1 - 2 - ▸ 3
T() → F() consumes 3; T sees end; T() returns	1 - 2 - 3 ▸
E() continues. next is end → loop ends. E() returns. Accept.	1 - 2 - 3 ▸



slide 7's tree · lit: created at this step

Every token consumed exactly once, in order. T() returned without touching the -: a procedure only consumes tokens that appear in its own rule, so the - was still there for E(). Both -s are consumed by the same E() call, in its loop — that is the left shape of slide 10.

The Python: the same procedures, now returning the tree.

E and F · T is E with '*' and atom

```
def expr(self):                                  # procedure E
    left = self.term()                           #   T()
    while self.at("OP", "-"):                    #   while next token is '-'
        self.expect("OP", "-")                   #     consume '-'
        right = self.term()                      #     T()
        left = ast.BinOp(left=left, op=ast.Sub(), right=right)   # NEW: join leftward — slide 10's shape
    return left                                  # NEW: hand back the subtree

def atom(self):                                  # procedure F
    if self.at("INT"):
        return ast.Constant(value=int(self.expect("INT").text))   # consume INT; NEW: return a leaf
    self.expect("OP", "("); e = self.expr(); self.expect("OP", ")")  # consume '(' ; E() ; consume ')'
    return e                                     # no node for parentheses: the shape already has them
```

the three helpers: look, test, consume-or-fail

```
def peek(self):              return self.toks[self.i]
def at(self, kind, text=None):
    t = self.peek();         return t.kind == kind and (text is None or t.text == text)
def expect(self, kind, text=None):
    if not self.at(kind, text):
        t = self.peek();     raise ParseError(f"line {t.line}, col {t.col}: expected {text or kind!r}")
    self.i += 1;             return self.toks[self.i - 1]
```

Slide 18's procedures, line for line, plus one thing: each procedure *returns* the tree for what it read, and `E()` joins the pieces leftward as its loop goes round — so the AST is `(1 - 2) - 3` even though slide 16's rewritten grammar leans right. `expect` is `consume`; it has the token, so a syntax error knows its line and column.

More operators means more levels, not more ideas: Python's expressions have eight (`or`, `and`, `not`, comparisons, `+ -`, `* // %`, unary `-`, atoms). Slide 12 gives one nonterminal per level; slides 17–18 give one procedure per nonterminal; every one is `expr` with a different operator set.

Monday and Wednesday.

Monday: Midterm 1, this room, one handwritten page of notes. Lectures 1–6. Every question is a doc-cam exercise you have already done: regex → NFA → DFA; derivation + parse tree; disambiguate a grammar; remove left recursion; trace `E()` on a string. **Redo today's three cold.**

Wednesday: Parsing II — LL(1): which grammars one token of lookahead can handle, and how to tell (FIRST and FOLLOW); the other direction, bottom-up parsing, which never has to choose a production, is untroubled by left recursion, and is what parser generators do; statements and the indentation grammar.

PA2, due Wednesday: the handout's grammar is slide 12's recipe with eight levels, already in the loop form; your parser is slides 17–18, one procedure per level; the oracle is `ast.dump(yours) == ast.dump(ast.parse(src))`. Thursday's section is a PA2 lab.